

Data Mining Complete Exam Notes

CT 72502 (Elective I) · BEI / BEX IV-I · TU, IOE · tiers over 23 papers, 2069–2082 BS

4. Association Analysis

10 hrs · in 23/23 papers · 5 topics, 3 TOP tier · **heaviest numericals in the syllabus** → Ch 4 companion.

Weight of the chapter:

- FP-Growth / FP-Tree **TOP 17/23** → **2nd most-asked topic in the whole subject.**
- Frequent Itemset & Apriori **TOP 14/23** → almost always a full numerical.
- Basics (support, confidence, market basket) **TOP 10/23**
- Sequential / Subgraph / Infrequent patterns **HOT 5/23** · Categorical attributes **PIN 3/23**

4.1 Basics and Algorithms

What is association analysis? Explain its importance in market-basket analysis. **2+5 10 2+6**

TOP 10/23 82 Bh, 80 Ba, 75 Ch, 75 Ash, 74 Ch, 73 Ch, 72 Ch, 72 Ka, 70 Ch, 70 Asa

Definitions

- **Itemset**: a set of 1 or more items. k -itemset = contains k items.
- **Transaction T** : a set of items bought together. D = the whole database.
- **Association rule**: an implication $A \Rightarrow B$ where $A \subset I$, $B \subset I$, and $A \cap B = \emptyset$.
- **Association analysis**: finding all rules that satisfy a minimum support and a minimum confidence.

The 2 measures (learn both definitions exactly)

$\sigma(X)$ = support count of X (how many transactions contain it), $|D|$ = total transactions.

$$\text{sup}(A \Rightarrow B) = P(A \cup B) = \frac{\sigma(A \cup B)}{|D|}$$

$$\text{conf}(A \Rightarrow B) = P(B|A) = \frac{\sigma(A \cup B)}{\sigma(A)}$$

- **Support** → how *frequent*. Filters out rules with too little evidence.
- **Confidence** → how *reliable*. Strength of the implication.
- A rule is **strong** if it meets *both* thresholds.
- **Support is symmetric** ($A \cup B$), **confidence is not** ($A \Rightarrow B \neq B \Rightarrow A$).

Market basket analysis 75 Ch short note

- Analyses which items customers put in the basket *together*.
- Classic result: {nappies} $\Rightarrow$ {beer}.
- **Uses**: shelf layout, cross-selling, product bundling, catalogue design, promotion planning, recommendation ("customers who bought this also bought...").
- **Actionable both ways**: put them together to increase convenience, or far apart to force the customer past other shelves.

2-step process (every algorithm does this)

1. **Find all frequent itemsets**: those with support $\geq$ min_sup. **This is the expensive step** and is what Apriori / FP-Growth optimise.
2. **Generate strong rules** from them: for each frequent itemset l , take every non-empty proper subset s and output $s \Rightarrow (l - s)$ if $\frac{\text{sup_count}(l)}{\text{sup_count}(s)} \geq \text{min_conf}$.
3. A k -itemset yields $2^k - 2$ candidate rules.

Applications beyond retail 80 Ba asks where it is applicable

- Web usage / click-stream mining, medical symptom-disease co-occurrence, **gene co-occurrence** (Ch 1), fraud detection, intrusion detection, text mining.

Pattern evaluation: why support-confidence is not enough 8

70 Ch “Why is pattern evaluation important? Explain the statistical based measures used for measuring interestingness.” **Full worked example in the companion.**

The problem

- A rule can clear both thresholds and still be **completely misleading**.
- **Classic case:** 10,000 txns, 6,000 contain *game*, 7,500 contain *video*, 4,000 contain both.
 - $support = 4000/10000 = 40\%$ (passes)
 - $confidence(game \Rightarrow video) = 4000/6000 = 66\%$ (passes)
 - But $P(video) = 75\%$ on its own, **higher than 66%**.
 - Buying a game makes you *less* likely to buy a video. The “strong” rule is **negatively correlated**.
- Cause: **confidence ignores the support of the consequent**.

Correlation measures verified/added

$$lift(A, B) = \frac{P(A \cup B)}{P(A)P(B)} = \frac{conf(A \Rightarrow B)}{sup(B)}$$

- > 1 positively correlated · < 1 negative · $= 1$ independent.
- Above: $lift = 0.40/(0.60 \times 0.75) = 0.89 < 1 \rightarrow$ caught.
- $\chi^2 = \sum \frac{(observed - expected)^2}{expected}$
- Above: $\chi^2 = 555.6$, and observed $<$ expected $\rightarrow$ negative correlation.

4 null-invariant measures

- $all_conf(A, B) = \frac{sup(A \cup B)}{\max\{sup(A), sup(B)\}} = \min\{P(A|B), P(B|A)\}$
- $max_conf(A, B) = \max\{P(A|B), P(B|A)\}$
- $Kulc(A, B) = \frac{1}{2}(P(A|B) + P(B|A))$
- $cosine(A, B) = \frac{sup(A \cup B)}{\sqrt{sup(A) \times sup(B)}}$
- **Null-invariant** = unaffected by the number of transactions containing *neither* item. **lift** and χ^2 are **NOT null-invariant**, so they distort on sparse data.
- Use **Kulc** + **imbalance ratio** together for the clearest reading.

Why a 100 % confidence rule is uninteresting, and why 99 % is 4

80 Ba “Generally we are more interested in rules with high confidence. However often we will not be interested in rules with a confidence of 100 %. Why? Then specifically explain why rules with 99 % confidence may be interesting.” verified/added

100 %: true by construction, so it teaches nothing

- $conf(A \Rightarrow B) = 100\%$ means **every** transaction holding *A* also holds *B*, no exceptions anywhere in *D*.
- Almost always that is a **property of the schema, not of the customers** $\rightarrow$ already known before mining started:
 - **Definitional:** $\{pregnant\} \Rightarrow \{female\}$, $\{Corolla\} \Rightarrow \{car\}$.
 - **Taxonomic:** $\{Kathmandu\} \Rightarrow \{Nepal\}$, item $\rightarrow$ its own category.
 - **Subset artefact:** $\{milk, bread\} \Rightarrow \{milk\}$. Consequent already sits inside the antecedent’s itemset.
 - **Bundled by a business rule:** free gift always shipped with the product, so the pair can never come apart.
- **No headroom, so no action.** You cannot lift sales of *B* by promoting *A*: everyone who takes *A* takes *B* already. Cross-selling, shelf layout and recommendation all have nothing to work with.
- Such rules are also generated in bulk (every frequent itemset spawns its own subset rules) and **bury the genuine findings**. Fix: a maximum support threshold, or drop rules whose consequent is implied by the antecedent.

99 %: real regularity, and the 1 % is the payload

- A definitional rule cannot have exceptions. So **99 % proves the dependency is empirical**, discovered from behaviour rather than built into the data dictionary.
- It is still near-deterministic $\rightarrow$ strong enough to predict on, unlike a 60 % rule.
- **The interest is in the 1 % that breaks it**, and it reads two ways. Give both:
 1. **The exceptions are wrong** $\rightarrow$ candidate **data-quality defects**: mis-keyed records, typos, a faulty sensor, a duplicated ID. Near-100 % rules are used exactly this way to discover integrity constraints and clean a warehouse (Ch 2).
 2. **The exceptions are real** $\rightarrow$ candidate **anomalies**: fraud, intrusion, a defective batch, a customer segment behaving unlike every other. This is the association-rule route into anomaly detection (Ch 6).
- Practical value too: 1 % is small but non-empty, so the customers who take *A* without *B* are a **precisely targetable list**.
- **One line worth memorising:** a 100 % rule tells you about the *schema*, a 99 % rule tells you about the *world*, and its exceptions are where the interesting cases live.

4.2 Frequent Itemset Pattern & Apriori Principle

Explain the Apriori algorithm / principle with example. 2+6 10 8

TOP 14/23 82 Bh, 81 Bh, 80 Bh, 80 Ba, 79 Bh, 79 Ba, 76 Ch, 76 Ash, 74 Ash, 73 Ch, 71 Ch, 71 Shr, 70 Ch, 69 Ch **Numerical in 7 of those** → companion.

The Apriori property (state this, it is the answer)

- All non-empty subsets of a frequent itemset must also be frequent.
- Contrapositive, and the one actually used: **if an itemset is infrequent, all of its supersets are infrequent** → prune them without counting.
- This is **anti-monotonicity**: support never increases as an itemset grows.
- **Effect**: turns a 2^n brute-force search into something tractable. 71 Ch asks exactly this, 10.

Brute force, for contrast

- n items → $2^n - 1$ candidate itemsets, each needing a database scan.
- Apriori prunes the lattice level by level instead.

Algorithm

1. Scan D , count each item → keep those $\geq \text{min_sup}$ → L_1 .
2. Repeat for $k = 2, 3, \dots$ until L_{k-1} is empty:
 - (a) **Join**: $C_k = L_{k-1} \bowtie L_{k-1}$. Join 2 itemsets only if their **first $k - 2$ items are identical** (keeps items sorted, avoids duplicates).
 - (b) **Prune**: drop any candidate having any $(k - 1)$ -subset not in L_{k-1} .
 - (c) **Count**: scan D once, count the survivors.
 - (d) $L_k =$ candidates meeting min_sup.
3. Answer = $\bigcup_k L_k$.

Then generate rules

- For each frequent l , for each non-empty proper subset s : output $s \Rightarrow (l - s)$ if $\frac{\text{sup}(l)}{\text{sup}(s)} \geq \text{min_conf}$.
- **When no rule can exist at any confidence** 79 Ba 3 **verified/added**: a rule needs a *proper* non-empty subset, so it needs a frequent itemset of size ≥ 2 . If $L_2 = \emptyset$ (every frequent itemset is a single item), then **no rule exists for any confidence threshold, including 0 %** → lowering min_conf cannot help, only lowering min_sup can. Check L_2 before answering, never the confidence value.

Advantages

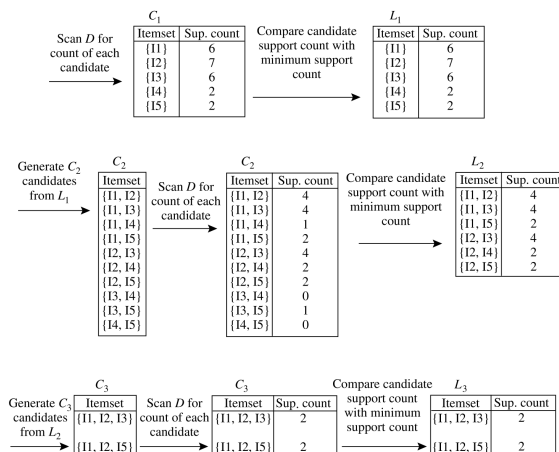
- Simple, easy to implement and to explain.
- Uses a level-wise search that is easy to parallelise.
- The pruning is **provably safe** (no frequent itemset is ever lost).

Disadvantages (the setup for FP-Growth)

- **Generates a huge number of candidates**. 10^4 frequent 1-itemsets → about 10^7 candidate 2-itemsets.
- **Requires repeated database scans** — one per level, so $k + 1$ scans for a k -itemset. Very costly on disk-resident data.
- Struggles with long patterns and with low min_sup.

The level-wise run, drawn

- H&K's 9-transaction example, min count 2: $C_1 \rightarrow L_1$, join to C_2 , scan, L_2 , join and **prune** to C_3 , scan, L_3 .
- Only 2 candidates survive into C_3 : every other 3-itemset has an infrequent 2-subset, so it is dropped **without a scan**. That is the Apriori property at work.
- This is the shape every numerical in the companion writes out. H&K Fig 6.2.



4.3 FP-Growth and the FP-Tree

What is FP-Growth? Explain with example. How does it overcome Apriori's problems? 2+5 8 10 2+6

TOP 17/23 82 Bh, 81 Bh, 81 Ba, 80 Bh, 79 Bh, 79 Ba, 78 Bh, 75 Ch, 75 Ash, 74 Ch, 74 Ash, 73 Ch, 73 Shr, 72 Ka, 71 Shr, 70 Asa, 69 Ch **The single most-asked topic in this chapter.**

Key idea

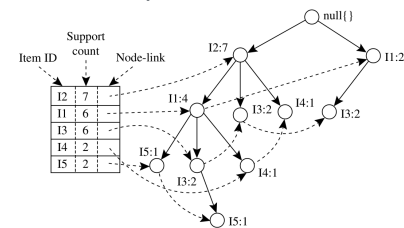
- **Generate NO candidates at all.**
- Compress D into a **frequent-pattern tree (FP-tree)**, then mine that tree by **divide and conquer**.
- Only **2 database scans** in total, against Apriori's $k + 1$.

Step 1: build the FP-tree

- Scan 1:** count every item. Discard the infrequent ones.
 - Sort the frequent items by **descending support** → the **F-list**. **Break ties consistently** (alphabetically) and say so.
 - Scan 2:** for each transaction, keep only frequent items, **reorder them into F-list order**, and insert that path into the tree.
 - Share a prefix with an existing path → just increment those counts.
 - Otherwise create new nodes with count 1.
 - Maintain a **header table**: one entry per frequent item, with a **node-link** chain threading every occurrence of that item in the tree.
- **Why descending order:** the most common items sit near the root, so paths share prefixes and the tree stays small.

Step 2: mine the tree (FP-Growth)

- Work **bottom-up** through the header table, ie **least** frequent item first.
- For item α , follow its node-links to collect its **conditional pattern base** = every prefix path leading to α , each carrying α 's count at that node.
- Build the **conditional FP-tree** from that base, keeping only items that are still frequent *within* it.
- Recurse** on the conditional tree, concatenating α to each pattern found.
- If a conditional tree is a **single path**, just enumerate all subsets of that path.



A finished FP-tree: **null** root, **item:count** nodes, and the header table (item, support) whose dashed **node-links** thread every occurrence of an item. Step 2 walks those links. H&K Fig 6.7.

Apriori vs FP-Growth (asked as a direct comparison 4 times)

	Apriori	FP-Growth
Candidates	Generates them	None
DB scans	$k + 1$	2
Strategy	Breadth-first, level-wise	Depth-first, divide & conquer
Structure	Hash tree	FP-tree (prefix tree)
Memory	Low	Tree must fit in memory
Speed	Slower	Roughly an order faster

- **FP-Growth's weakness:** if the tree does not fit in memory, or the data is very sparse so paths barely share prefixes, the advantage disappears.

4.4 Handling Categorical Attributes

How do you handle categorical attributes in data mining? Issues and fixes. 3+3 2+3+3 4

PIN 3/23 79 Bh, 73 Shr, 72 Ch

The problem

- Association analysis assumes **asymmetric binary** items (present / absent).
- Real tables hold categorical (Gender, Region) and continuous (Age, Income) attributes.

Handling: binarisation

- Convert each (**attribute, value**) pair into one **binary item**.
- Eg Gender → items Gender=Male, Gender=Female.
- Continuous attributes must be **discretised** first → Ch 2, §2.2, then binarised.

Issues that arise (this is what 73 Shr wants)

- **Too many items**: an attribute with many values explodes the item count. **Fix**: group rare values into an “Other” category.
- **Many items are infrequent**: rare values never reach min_sup, so patterns involving them are lost. **Fix**: lower min_sup, or use a per-item threshold.
- **Some items are far too frequent**: eg Gender=Male at ≈50% produces masses of trivial redundant rules. **Fix**: set a maximum support threshold too.
- **Redundancy**: values of one attribute are **mutually exclusive**, so an itemset containing Gender=Male and Gender=Female has support 0 by construction. **Fix**: never generate candidates that pair 2 values of the same attribute.
- **Loss of ordering** for ordinal attributes after binarisation.

4.5 Sequential, Subgraph and Infrequent Patterns

Sequential patterns, subsequences, subgraph patterns 3+3 2 3

HOT 5/23 79 Bh, 76 Ch, 76 Ash, 73 Ch, 72 Ka Subsequence and subgraph enumeration are numericals → companion.

Sequential patterns

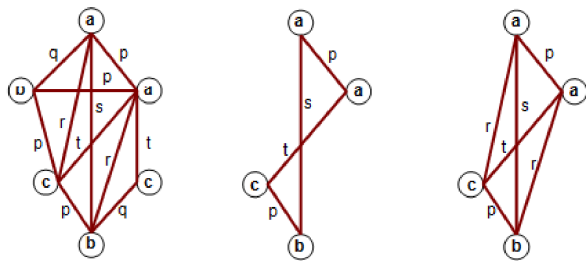
- A **sequence** is an *ordered* list of **elements**; each element is an unordered *set* of items. $s = \langle e_1 e_2 \dots e_n \rangle$.
- **Length** = number of *items*; a *k*-sequence has *k* items. **Not the number of elements** — this is the usual mistake.
- **Subsequence**: $\langle a_1 \dots a_m \rangle$ is a subsequence of $\langle b_1 \dots b_n \rangle$ if there exist $i_1 < i_2 < \dots < i_m$ with $a_1 \subseteq b_{i_1}, \dots, a_m \subseteq b_{i_m}$.
 - **Order across elements must be preserved; within an element it need not be.**
- **Support** of a sequence = fraction of data sequences containing it.
- **Algorithms**: AprioriAll 76 Ash short note, GSP, SPADE, PrefixSpan. AprioriAll applies the same anti-monotone pruning to sequences.
- **Uses**: web click-streams, purchase sequences over time, DNA, telephone call patterns.

Subgraph patterns 79 Bh 2

- Data = a collection of **graphs** (molecules, web fragments, social networks).
- A **frequent subgraph** is one appearing in at least min_sup of them.
- **Candidate generation by growing**, 2 schemes:
 - **Vertex growing**: join 2 graphs sharing a common subgraph, add 1 vertex.
 - **Edge growing**: join 2 graphs sharing a core, add 1 edge. **Preferred** — it generates fewer candidates.
- **The hard part is graph isomorphism**: deciding whether 2 drawings are the same graph. Handled with a canonical labelling.
- Algorithms: AGM, FSG, gSpan.

Infrequent / negative patterns

- A **negative pattern** is a set of items that appear together *far less* often than independence would predict.
- Eg {Coke, Pepsi} — customers buy one or the other.
- Useful for detecting substitute products and for fraud (a combination that should never occur).
- **Hard because** support is anti-monotone, so “infrequent” cannot be pruned the same way.



(a) Labeled Graph

(b) Subgraph

(c) Induced Subgraph

Subgraph terms, drawn

(a) a labelled graph; (b) a **subgraph**: some of its vertices and edges; (c) an **induced subgraph**: some vertices and every edge between them. Candidate generation grows these one vertex or one edge at a time. Mishra's Ch 4 notes, p9.

4.6 PYQ Mapping

N = solved in the Ch 4 Numerical companion.

4.1 Basics

1. **TOP 7/23** Association analysis + importance in market-basket analysis. **2+5** 72 Ka **10** 72 Ch **2+6** 80 Ba
2. **PIN 2/23** Why is association analysis required? + Apriori principle w/ example. **2+6** 75 Ash, 74 Ch
3. **PIN 1/23** Importance of SUPPORT and CONFIDENCE + FP-Growth w/ example. **10** 72 Ch
4. **N PIN 1/23** Why is pattern evaluation important? + statistical interestingness measures. **8** 70 Ch
5. **PIN 1/23** Why 100% confidence is uninteresting but 99% is. **4** 80 Ba
6. **PIN 1/23** Short note: Market Basket Analysis. **4** 75 Ch

4.2 Apriori

7. **N** Apriori, min support count 4, confidence 80%, 7 transactions. **2+7** 80 Bh
8. **N** Candidate + large itemsets, min support 2. **4+5** 80 Ba
9. **N** Derive association rules, min sup 50%, min conf 80%. **8** 79 Bh
10. **N** Fast-food orders, min sup 2/9, min conf 7/9; frequent k -itemsets + strong rules. **5+3** 76 Ch
11. **N** L-itemsets at $L = 3$ w/ Apriori + F-list w/ FP-Growth. **12** 76 Ash
12. **N** Candidate, frequent itemsets and rules, min sup 20%, min conf 80%. **8** 74 Ash
13. **N** Apriori w/ support 50%. **10** 70 Ch
14. **PIN 1/23** How does Apriori optimise the brute-force approach? **10** 71 Ch
15. **PIN 1/23** What are association rules? How does Apriori generate them? **10** 71 Shr

4.3 FP-Growth

16. **TOP 7/23** What is FP-Growth? Explain w/ example. **2+5** 80 Bh **8** 74 Ash **10** 71 Shr **2+6** 75 Ash **1+8** 72 Ka **2+5** 81 Ba
17. **N** Draw FP-tree for the given 5-transaction table. **4+4** 75 Ch
18. **N** Limitation of Apriori vs FP-Growth + $\{M, O, N, K, E, Y\}$ database, min sup 60%, min conf 80%. **2+7** 81 Ba
19. **N** Build FP-tree transaction by transaction + mine it, threshold 33.34% / 60%. **4+4** 78 Bh
20. **N** How does FP-Growth overcome Apriori? + generate the FP-tree. **2+8** 74 Ch
21. **N** FP-tree + association rules, support 30%, confidence 75%. **10** 73 Shr
22. **PIN 1/23** Short note: FP-Tree. **5** 79 Bh

4.4 / 4.5 Categorical, Sequential, Subgraph

23. **N** Handle categorical attributes + generate 4 subsequences from $\{\{2, 3, 5\}, \{6, 7, 8\}, \{9, 1\}, \{7, 4\}\}$. **3+3** 79 Bh
24. **PIN 1/23** Categorical data, issues, how to handle them. **2+3+3** 73 Shr
25. **N** List all 4-subsequences of $\{\{1, 3\}\{2\}\{2, 3\}\{4\}\}$ + draw candidate subgraphs by edge growing. **3+3** 76 Ch
26. **PIN 1/23** What are subgraph patterns? **2** 79 Bh
27. **PIN 3/23** Short notes: AprioriAll 76 Ash · Sequential pattern 72 Ka · Categorical data 72 Ch. **3** **4**

Papers added by the 2026-09 merge

The six papers of DM4.1BCT.pdf (2082 Bh, 2081 Bh, 2079 Ba, 2073 Ch, 2070 Asa, 2069 Ch), indexed here for the first time — the notes were built before that scan was merged. **N** = solved in this chapter's companion. **verified/added**

- **N** Support and confidence + derive rules, min sup 60%, min conf 80%, 5 transactions. **2+8** 82 Bh
- **N** Apriori with **confidence 65%** and min support 50%, 4 transactions. **2+6** 81 Bh

- **N** Apriori on a 7×9 **binary** table (min_sup 20%) + **can any rule exist at any confidence threshold?** 5+3 79 Ba
verified/added
- **N** Apriori in market-basket analysis, both thresholds 50%. 3+7 73 Ch
- Advantages of FP-Growth over Apriori + FP-Growth w/ example. 2+6 81 Bh
- How does FP-Growth generate frequent itemsets **without candidate generation?** 6 79 Ba
- Use of FP-Growth in market-basket analysis 73 Ch · FP-Growth in detail 70 Asa. 10 12
- What is frequent itemset mining? How do Apriori and FP-Growth optimise the brute-force approach? 15 69 Ch
- How can Apriori be used to find rules out of a frequent itemset? 7 69 Ch
- Association rules + their importance w/ example 70 Asa · short note **FP Growth** 82 Bh. 6 5
- Sequential pattern and sub-graph pattern w/ suitable example. 4+4 73 Ch

Coverage check

- All Ch 4 PYQs covered. **14 are numerical** → companion.
- **Gaps filled by research** verified/added :
 - The **null-invariant measures** (all_conf, max_conf, Kulc, cosine) and the game/video counter-example for 70 Ch: your notes define only support and confidence.
 - **Issues with categorical attributes** 73 Shr: the mutual-exclusion and too-frequent-item problems are in no source note.
 - **Edge vs vertex growing** for subgraphs.

Sources used for this chapter

- **Han & Kamber 3rd ed** ch 6 (frequent patterns, Apriori, FP-Growth, pattern evaluation) and ch 7 (advanced pattern mining). **Primary authority.**
- **DBK Lectures 12–14**, data mining_/Chapter-4 Association Analysis.pdf, Associations_Rule_mining.ppt, FP-Growth algorithm.ppt, FP Tree.pdf, Episode_Mining.PDF.

Abbreviations used in these notes

Only these are ever shortened. Anything not on this list is written out in full.

Short	Means	Short	Means
DM	data mining	DW	data warehouse
DB	database	DBMS	database management system
KDD	knowledge discovery in databases	DSS	decision support system
OLTP	on-line transaction processing	OLAP	on-line analytical processing
ROLAP	relational OLAP	MOLAP	multidimensional OLAP
HOLAP	hybrid OLAP	ETL	extract, transform, load
BI	business intelligence	ER	entity-relationship
attr(s)	attribute(s)	dim(s)	dimension(s)
multidim	multidimensional	txn(s)	transaction(s)
info	information	ops	operations
repo	repository	freq	frequent / frequency
config	configuration	no.	number
PCA	principal component analysis	SVD	singular value decomposition
SMC	simple matching coefficient	KNN	k -nearest neighbour
ANN	artificial neural network	POS	point of sale
FK	foreign key	UI	user interface
GIGO	garbage in, garbage out	w/, w/o	with, without
rm	remove	vs	versus
Eg	example / for example	+Fig	draw a figure
→	leads to / therefore	+	and

Exam-paper month codes (used in the year tags): Ba = Baishakh · Jth = Jestha · Asa = Ashar · Shr = Shrawan · Bh = Bhadra · Ash = Ashwin · Ka = Kartik · Mng = Mangsir · Po = Poush · Ma = Magh · Ch = Chaitra.

Tier chips: **TOP /23** = topic asked in 7 or more of the 17 papers · **HOT /23** = 4–6 papers · **PIN /23** = 1–3 papers but worth ≥ 8 marks. **Mark chips** such as **2+5** are the marks that question actually carried. verified/added marks anything researched and added that was not in the source notes.